

1. Selection Sort

Program:

```
#include <stdio.h>

void selectionSort(int arr[], int n)
{
    int i, j, min, temp;

    for (i = 0; i < n - 1; i++)
    {
        min = i;

        for (j = i + 1; j < n; j++)
        {
            if (arr[j] < arr[min])
                min = j;
        }

        temp = arr[i];
        arr[i] = arr[min];
        arr[min] = temp;
    }
}

int main()
{
    int n;

    printf("Enter array size: ");
    scanf("%d", &n);

    if (n == 0)
    {
        printf("[]");
        return 0;
    }

    int arr[n];

    printf("Enter array elements:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    selectionSort(arr, n);

    printf("\nSorted Array:\n");

    printf("[");
```

```

for (int i = 0; i < n; i++)
{
    printf("%d", arr[i]);

    if (i != n - 1)
        printf(", ");
}
printf("]");

return 0;
}

```

Output:

```

C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 2\1.exe
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 2> gcc "1.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 2> .\program.exe
Enter array size: 5
Enter array elements:
-5 -1 -3 -2 -4

Sorted Array:
[-5, -4, -3, -2, -1]

```

2. Selection Sort

Program:

```

#include <stdio.h>

void selectionSort(int arr[], int n)
{
    int i, j, min, temp;

    for (i = 0; i < n - 1; i++)
    {
        min = i;

        for (j = i + 1; j < n; j++)
        {
            if (arr[j] < arr[min])
                min = j;
        }

        temp = arr[i];
        arr[i] = arr[min];
        arr[min] = temp;
    }
}

```

```

int main()
{
    int n;

    printf("Enter array size: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter array elements:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    selectionSort(arr, n);

    printf("\nSorted Array:\n");

    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    printf("\nTime Complexity: O(n^2)");

    return 0;
}

```

Output:

```

C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 2\2.exe
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 2> gcc "2.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 2> .\program.exe
Enter array size: 6
Enter array elements:
5 2 9 1 5 6

Sorted Array:
1 2 5 5 6 9
Time Complexity: O(n^2)

```

3. Bubble Sort

Program:

```

#include <stdio.h>

void bubbleSort(int arr[], int n)
{
    int i, j, temp;
    int swapped;

    for (i = 0; i < n - 1; i++)

```

```

    {
        swapped = 0;

        for (j = 0; j < n - i - 1; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = 1;
            }
        }

        if (swapped == 0)
            break;
    }
}

int main()
{
    int n;

    printf("Enter array size: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter array elements:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    bubbleSort(arr, n);

    printf("\nSorted Array:\n");

    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    printf("\nTime Complexity: Best Case O(n), Worst Case O(n^2)");

    return 0;
}

```

Output:

```

C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 2\3.exe
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 2> gcc "3.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 2> .\program.exe
Enter array size: 5
Enter array elements:
64 25 12 22 11

Sorted Array:
11 12 22 25 64
Time Complexity: Best Case  $O(n)$ , Worst Case  $O(n^2)$ 

```

4. Insertion Sort

Program:

```
#include <stdio.h>
```

```

void insertionSort(int arr[], int n)
{
    int i, key, j;

    for (i = 1; i < n; i++)
    {
        key = arr[i];
        j = i - 1;

        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j--;
        }

        arr[j + 1] = key;
    }
}

```

```

int main()
{
    int n;

    printf("Enter array size: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter array elements:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);
}

```

```

insertionSort(arr, n);

printf("\nSorted Array:\n");

for (int i = 0; i < n; i++)
    printf("%d ", arr[i]);

printf("\nTime Complexity: O(n^2)");

return 0;
}

```

Output:

```

C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 2\4.exe
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 2> gcc "4.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 2> .\program.exe
Enter array size: 10
Enter array elements:
3 1 4 1 5 9 2 6 5 3

Sorted Array:
1 1 2 3 3 4 5 5 6 9
Time Complexity: O(n^2)

```

5. Missing Number

Program:

```
#include <stdio.h>
```

```

int findKthMissing(int arr[], int n, int k)
{
    int current = 1;
    int i = 0;

    while (k > 0)
    {
        if (i < n && arr[i] == current)
        {
            i++;
        }
        else
        {
            k--;
            if (k == 0)
                return current;
        }

        current++;
    }
}

```

```

    }

    return current;
}

int main()
{
    int n, k;

    printf("Enter array size: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter array elements:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    printf("Enter k: ");
    scanf("%d", &k);

    printf("\nKth Missing Number = %d", findKthMissing(arr, n, k));

    return 0;
}

```

Output:

```

C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 2\5.exe
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 2> gcc "5.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 2> .\program.exe
Enter array size: 5
Enter array elements:
2 3 4 7 11
Enter k: 5

Kth Missing Number = 9

```

6. Peak Element

Program:

```

#include <stdio.h>

int findPeak(int arr[], int n)
{
    int low = 0;
    int high = n - 1;

    while (low < high)

```

```

    {
        int mid = (low + high) / 2;

        if (arr[mid] < arr[mid + 1])
            low = mid + 1;
        else
            high = mid;
    }

    return low;
}

int main()
{
    int n;

    printf("Enter array size: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter array elements:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    int index = findPeak(arr, n);

    printf("\nPeak Index = %d", index);
    printf("\nPeak Element = %d", arr[index]);

    return 0;
}

```

Output:

```

C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 2\6.exe
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 2> gcc "6.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 2> .\program.exe
Enter array size: 4
Enter array elements:
1 2 3 1

Peak Index = 2
Peak Element = 3

```

7. String Matching

Program:

```
#include <stdio.h>
```

```

#include <string.h>

int findSubstring(char haystack[], char needle[])
{
    int n = strlen(haystack);
    int m = strlen(needle);

    for (int i = 0; i <= n - m; i++)
    {
        int j;

        for (j = 0; j < m; j++)
        {
            if (haystack[i + j] != needle[j])
                break;
        }

        if (j == m)
            return i;
    }

    return -1;
}

int main()
{
    char haystack[100];
    char needle[100];

    printf("Enter haystack: ");
    scanf("%s", haystack);

    printf("Enter needle: ");
    scanf("%s", needle);

    int index = findSubstring(haystack, needle);

    printf("\nIndex = %d", index);

    return 0;
}

```

Output:

```

C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 2\7.exe
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 2> gcc "7.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 2> .\program.exe
Enter haystack: sadbutsad
Enter needle: sad

Index = 0

```

8. Substring Search

Program:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main()
```

```
{
```

```
    int n;
```

```
    printf("Enter number of words: ");
```

```
    scanf("%d", &n);
```

```
    char words[n][100];
```

```
    printf("Enter the words:\n");
```

```
    for (int i = 0; i < n; i++)
```

```
        scanf("%s", words[i]);
```

```
    printf("\nSubstring Words:\n");
```

```
    int found = 0;
```

```
    for (int i = 0; i < n; i++)
```

```
    {
```

```
        for (int j = 0; j < n; j++)
```

```
        {
```

```
            if (i != j && strstr(words[j], words[i]) != NULL)
```

```
            {
```

```
                printf("%s\n", words[i]);
```

```
                found = 1;
```

```
                break;
```

```
            }
```

```
        }
```

```
    }
```

```
    if (!found)
```

```
        printf("No substring found");
```

```
    return 0;
```

```
}
```

Output:

```
C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 2\8.exe
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 2> gcc "8.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 2> .\program.exe
Enter number of words: 4
Enter the words:
mass
as
hero
superhero

Substring Words:
as
hero
```

9. Closest Pair

Program:

```
#include <stdio.h>
```

```
#include <math.h>
```

```
struct Point
```

```
{
```

```
    int x;
```

```
    int y;
```

```
};
```

```
int main()
```

```
{
```

```
    int n;
```

```
    printf("Enter number of points: ");
```

```
    scanf("%d", &n);
```

```
    struct Point p[n];
```

```
    printf("Enter the points (x y):\n");
```

```
    for (int i = 0; i < n; i++)
```

```
        scanf("%d %d", &p[i].x, &p[i].y);
```

```
    double minDist = 1e9;
```

```
    int a = 0, b = 1;
```

```
    for (int i = 0; i < n; i++)
```

```
    {
```

```

    for (int j = i + 1; j < n; j++)
    {
        double dx = p[i].x - p[j].x;
        double dy = p[i].y - p[j].y;
        double dist = sqrt(dx * dx + dy * dy);

        if (dist < minDist)
        {
            minDist = dist;
            a = i;
            b = j;
        }
    }
}

printf("\nClosest Pair: (%d, %d) and (%d, %d)\n",
    p[a].x, p[a].y, p[b].x, p[b].y);

printf("Minimum Distance = %.6lf\n", minDist);

return 0;
}

```

Output:

```

C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 2\9.exe
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 2> gcc "9.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 2> .\program.exe
Enter number of points: 4
Enter the points (x y):
1 2
4 5
7 8
3 1

Closest Pair: (1, 2) and (3, 1)
Minimum Distance = 2.236068

```

10. Convex Hull

Program:

```
#include <stdio.h>
```

```
struct Point
```

```

{
    float x;
    float y;
};

```

```

int orientation(struct Point p1, struct Point p2, struct Point p3)
{
    float value = (p2.y - p1.y) * (p3.x - p2.x) -
                  (p2.x - p1.x) * (p3.y - p2.y);

    if (value > 0)
        return 1;
    if (value < 0)
        return -1;
    return 0;
}

```

```

int main()
{
    int n;

    printf("Enter number of points: ");
    scanf("%d", &n);

    struct Point p[n];

    printf("Enter the points (x y):\n");

    for (int i = 0; i < n; i++)
        scanf("%f %f", &p[i].x, &p[i].y);

    printf("\nConvex Hull Points:\n");

    for (int i = 0; i < n; i++)
    {
        for (int j = i + 1; j < n; j++)
        {
            int pos = 0, neg = 0;

            for (int k = 0; k < n; k++)
            {
                if (k == i || k == j)
                    continue;

                int o = orientation(p[i], p[j], p[k]);

                if (o > 0)
                    pos++;
                else if (o < 0)
                    neg++;
            }

            if (pos == 0 || neg == 0)

```

```

        {
            printf("(%.1f, %.1f) (%.1f, %.1f)\n",
                p[i].x, p[i].y,
                p[j].x, p[j].y);
        }
    }
}

return 0;
}

```

Output:

```

C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 2\10.exe
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 2> gcc "10.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 2> .\program.exe
Enter number of points: 8
Enter the points (x y):
10 0
11 5
5 3
9 3.5
15 3
12.5 7
6 6.5
7.5 4.5

Convex Hull Points:
(10.0, 0.0) (5.0, 3.0)
(10.0, 0.0) (15.0, 3.0)
(5.0, 3.0) (6.0, 6.5)
(15.0, 3.0) (12.5, 7.0)
(12.5, 7.0) (6.0, 6.5)

```

11. Convex Hull

Program:

```
#include <stdio.h>
```

```
struct Point
```

```
{
    int x;
    int y;
};
```

```
int orientation(struct Point p1, struct Point p2, struct Point p3)
```

```
{
    int value = (p2.y - p1.y) * (p3.x - p2.x) -
                (p2.x - p1.x) * (p3.y - p2.y);
}
```



```

    }

    return 0;
}

```

Output:

```

C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 2\11.exe
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 2> gcc "11.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 2> .\program.exe
Enter number of points: 5
Enter the points (x y):
1 1
4 6
8 1
0 0
3 3

Convex Hull:
(4,6) (8,1)
(4,6) (0,0)
(8,1) (0,0)

```

12. Travelling Salesman

Program:

```

#include <stdio.h>
#include <math.h>

```

```

#define MAX 10

```

```

double distance(double x1, double y1, double x2, double y2)
{
    return sqrt((x2 - x1) * (x2 - x1) + (y2 - y1) * (y2 - y1));
}

```

```

void swap(int *a, int *b)
{
    int t = *a;
    *a = *b;
    *b = t;
}

```

```

void permute(int arr[], int l, int r, double x[], double y[],
             double *minDist, int bestPath[])
{
    if (l == r)
    {
        double dist = 0;

```

```

    int prev = 0;

    for (int i = 0; i < r; i++)
    {
        dist += distance(x[prev], y[prev], x[arr[i]], y[arr[i]]);
        prev = arr[i];
    }

    dist += distance(x[prev], y[prev], x[0], y[0]);

    if (dist < *minDist)
    {
        *minDist = dist;

        for (int i = 0; i < r; i++)
            bestPath[i] = arr[i];
    }

    return;
}

for (int i = 1; i <= r; i++)
{
    swap(&arr[l], &arr[i]);
    permute(arr, l + 1, r, x, y, minDist, bestPath);
    swap(&arr[l], &arr[i]);
}
}

int main()
{
    int n;

    printf("Enter number of cities: ");
    scanf("%d", &n);

    double x[MAX], y[MAX];

    printf("Enter city coordinates:\n");

    for (int i = 0; i < n; i++)
        scanf("%lf %lf", &x[i], &y[i]);

    int arr[MAX];
    int bestPath[MAX];

    for (int i = 1; i < n; i++)

```

```

        arr[i - 1] = i;

double minDist = 1e9;

permute(arr, 0, n - 2, x, y, &minDist, bestPath);

printf("\nShortest Distance = %.6lf\n", minDist);

printf("Shortest Path:\n");
printf("0 ");

for (int i = 0; i < n - 1; i++)
    printf("%d ", bestPath[i]);

printf("0\n");

return 0;
}

```

Output:

```

C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 2\12.exe
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 2> gcc "12.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 2> .\program.exe
Enter number of cities: 4
Enter city coordinates:
1 2
4 5
7 1
3 6

Shortest Distance = 10.128990
Shortest Path:
0 1 3 -130199552 0

```

13. Assignment Problem

Program:

```

#include <stdio.h>

void swap(int *a, int *b)
{
    int t = *a;
    *a = *b;
    *b = t;
}

int totalCost(int assign[], int cost[][10], int n)
{

```

```

    int sum = 0;

    for (int i = 0; i < n; i++)
        sum += cost[i][assign[i]];

    return sum;
}

void permute(int assign[], int l, int r, int cost[][10], int n, int *minCost, int best[])
{
    if (l == r)
    {
        int current = totalCost(assign, cost, n);

        if (current < *minCost)
        {
            *minCost = current;

            for (int i = 0; i < n; i++)
                best[i] = assign[i];
        }

        return;
    }

    for (int i = l; i <= r; i++)
    {
        swap(&assign[l], &assign[i]);
        permute(assign, l + 1, r, cost, n, minCost, best);
        swap(&assign[l], &assign[i]);
    }
}

int main()
{
    int n;

    printf("Enter number of workers/tasks: ");
    scanf("%d", &n);

    int cost[10][10];

    printf("Enter cost matrix:\n");

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);
    }
}

```

```

    }

    int assign[10], best[10];

    for (int i = 0; i < n; i++)
        assign[i] = i;

    int minCost = 1000000;

    permute(assign, 0, n - 1, cost, n, &minCost, best);

    printf("\nOptimal Assignment:\n");

    for (int i = 0; i < n; i++)
        printf("Worker %d -> Task %d\n", i + 1, best[i] + 1);

    printf("Total Cost = %d\n", minCost);

    return 0;
}

```

Output:

```

C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 2\13.exe
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 2> gcc "13.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 2> .\program.exe
Enter number of workers/tasks: 3
Enter cost matrix:
3 10 7
8 5 12
4 6 9

Optimal Assignment:
Worker 1 -> Task 3
Worker 2 -> Task 2
Worker 3 -> Task 1
Total Cost = 16

```

14. Knapsack Problem

Program:

```

#include <stdio.h>

int main()
{
    int n;

    printf("Enter number of items: ");
    scanf("%d", &n);
}

```

```

int weight[n], value[n];
int capacity;

printf("Enter weights:\n");
for (int i = 0; i < n; i++)
    scanf("%d", &weight[i]);

printf("Enter values:\n");
for (int i = 0; i < n; i++)
    scanf("%d", &value[i]);

printf("Enter capacity: ");
scanf("%d", &capacity);

int maxVal = 0;
int bestMask = 0;

for (int mask = 0; mask < (1 << n); mask++)
{
    int totalWeight = 0;
    int totalValue = 0;

    for (int i = 0; i < n; i++)
    {
        if (mask & (1 << i))
        {
            totalWeight += weight[i];
            totalValue += value[i];
        }
    }

    if (totalWeight <= capacity && totalValue > maxVal)
    {
        maxVal = totalValue;
        bestMask = mask;
    }
}

printf("\nOptimal Selection: ");

for (int i = 0; i < n; i++)
{
    if (bestMask & (1 << i))
        printf("%d ", i);
}

printf("\nTotal Value = %d\n", maxVal);

```

```
    return 0;  
}
```

Output:

```
C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 2\14.exe  
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 2> gcc "14.c" -o program  
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 2> .\program.exe  
Enter number of items: 3  
Enter weights:  
2 3 1  
Enter values:  
4 5 3  
Enter capacity: 4  
  
Optimal Selection: 1 2  
Total Value = 8
```